

SQL Filters

Refining Query Results with the WHERE Clause

Lab Manual — Lab #1

Course	Database Systems
Topic	Filtering rows: WHERE, comparison & logical operators, BETWEEN, IN, LIKE, IS NULL, ORDER BY, LIMIT
Duration	One lab session (approx. 3 hours)
Tools	MySQL 8.x, MySQL Workbench / Command Line Client
Prerequisites	Basic SQL: CREATE / INSERT / SELECT, primary keys, normalization
Next Lab	SQL Joins (Lab #2)

1. Learning Objectives

By the end of this lab, students will be able to:

- 1 Filter rows using the **WHERE** clause with comparison operators (=, !=, <, >, <=, >=).
- 2 Combine multiple conditions using **AND**, **OR**, and **NOT**, applying correct operator precedence.
- 3 Use **BETWEEN** for inclusive range checks on numbers and dates.
- 4 Use **IN** to test membership against a list of values.
- 5 Use **LIKE** with wildcards (%) and (_) for pattern matching on text columns.
- 6 Correctly handle NULL values using **IS NULL** and **IS NOT NULL**.
- 7 Sort query results using **ORDER BY** (ASC/DESC, multi-column).
- 8 Restrict the number of rows returned with **LIMIT**.
- 9 Combine all of the above to answer realistic business questions on a single table.

Outcome

Each student will independently write and run filter queries on a given table, producing a single .sql file with all answers, ready to be pushed to their GitHub repository.

2. Session Plan (3 Hours)

Time	Activity	Mode
0:00 – 0:10	Introduction & lab schema setup (run provided script)	Setup
0:10 – 0:35	WHERE clause, comparison operators, simple filters	Lecture + Demo
0:35 – 1:00	Logical operators: AND, OR, NOT and operator precedence	Lecture + Demo
1:00 – 1:30	Lab Tasks Part A — Basic filters and logical operators	Hands-on
1:30 – 1:40	Short break	—
1:40 – 2:05	BETWEEN, IN, LIKE patterns, IS NULL	Lecture + Demo
2:05 – 2:25	ORDER BY (single + multi-column) and LIMIT	Lecture + Demo
2:25 – 2:55	Lab Tasks Part B — Range, list, pattern, NULL, sorting	Hands-on
2:55 – 3:00	Wrap-up, common mistakes, deliverables	Discussion

3. The Lab Schema — Employee Database

This lab uses a single **Employee** table loaded with realistic data. Run the setup script below in MySQL **before** starting the exercises. Lab #2 will build on this schema by adding related tables — for now, everything happens on this one table.

3.1 Table Structure

Column	Type	Description
EmpID	INT (PK)	Unique employee identifier
EmpName	VARCHAR(50)	Full name of the employee
Gender	CHAR(1)	'M' or 'F'
Salary	DECIMAL(10,2)	Monthly salary in PKR
HireDate	DATE	Date the employee joined the company
City	VARCHAR(30)	Home city (may be NULL)
JobTitle	VARCHAR(40)	Designation (e.g. 'Software Engineer')
DeptName	VARCHAR(40)	Department name (denormalized for this lab)

3.2 Setup Script — Run this First

```
CREATE DATABASE IF NOT EXISTS filters_lab;
USE filters_lab;

DROP TABLE IF EXISTS Employee;

CREATE TABLE Employee (
    EmpID      INT PRIMARY KEY,
    EmpName    VARCHAR(50) NOT NULL,
    Gender     CHAR(1),
    Salary     DECIMAL(10,2),
    HireDate   DATE,
    City       VARCHAR(30),
    JobTitle   VARCHAR(40),
    DeptName   VARCHAR(40)
);

INSERT INTO Employee VALUES
(101, 'Ali Khan',      'M', 120000, '2018-03-15', 'Lahore', 'Senior
Engineer', 'Engineering'),
(102, 'Sara Iqbal',    'F', 95000, '2019-06-01', 'Lahore', 'Software
Engineer', 'Engineering'),
(103, 'Hamza Raza',    'M', 85000, '2020-01-20', 'Karachi', 'Software
Engineer', 'Engineering'),
(104, 'Ayesha Noor',   'F', 110000, '2017-11-10', 'Karachi', 'Marketing Lead',
'Marketing'),
(105, 'Bilal Ahmed',   'M', 70000, '2021-04-05', 'Karachi', 'Marketing Exec',
'Marketing'),
(106, 'Fatima Sheikh', 'F', 90000, '2019-09-12', 'Islamabad', 'Accountant',      'Finance'),
(107, 'Usman Tariq',   'M', 78000, '2022-02-18', 'Islamabad', 'Accountant',
'Finance'),
(108, 'Maira Javed',   'F', 115000, '2016-07-22', 'Lahore', 'Research Lead', 'Research'),
(109, 'Zain Abbas',    'M', 60000, '2023-01-09', 'Lahore', 'Research
Analyst', 'Research');
```

```
(110,'Nida Yousaf',      'F', 72000,'2022-08-30',NULL,      'Research Analyst','Research'),
(111,'Adeel Akhtar',    'M', 88000,'2020-05-14','Lahore', 'QA Engineer', 'Engineering'),
(112,'Sana Malik',      'F',102000,'2018-12-01','Karachi', 'Sales Manager', 'Sales'),
(113,'Talha Hussain',   'M', 65000,'2023-07-18','Islamabad','Sales Exec',      'Sales'),
(114,'Mehwish Anwar',   'F', 80000,'2021-10-25','Lahore', 'HR Officer',      'HR'),
(115,'Imran Shafi',     'M',125000,'2015-04-30',NULL, 'Director',      'Engineering');
```

Note the data quirks

Nida (110) and **Imran (115)** have NULL city — useful for IS NULL practice. Salaries range from 60,000 to 125,000 — handy for range queries. Hire dates span 2015–2023 — useful for date-based BETWEEN exercises. Department and job title columns are denormalized on purpose so this lab is self-contained.

4. The WHERE Clause

The **WHERE** clause filters rows from the source table. Logically, it runs *after* FROM but *before* SELECT, so it can reference any column in the table — but not column aliases defined in the SELECT list.

```
SELECT column_list
FROM table_name
WHERE condition;
```

4.1 Comparison Operators

Operator	Meaning	Example
=	Equal to	Salary = 95000
!= or <>	Not equal to	City != 'Lahore'
<	Less than	Salary < 80000
>	Greater than	Salary > 100000
<=	Less than or equal	HireDate <= '2020-12-31'
>=	Greater than or equal	Salary >= 90000

Class Example 1 — Employees earning over 100,000:

```
SELECT EmpID, EmpName, Salary
FROM Employee
WHERE Salary > 100000;
```

Expected output (4 rows): Ali Khan, Ayesha Noor, Maira Javed, Sana Malik, Imran Shafi.

Class Example 2 — Employees not in Lahore:

```
SELECT EmpName, City
FROM Employee
WHERE City != 'Lahore';
```

NULL trap

The query above will **not** return Nida or Imran, even though they are obviously not in Lahore — they have NULL city. NULL comparisons always return UNKNOWN, never TRUE. We will fix this with IS NULL in section 4.6.

4.2 Logical Operators: AND, OR, NOT

Combine multiple conditions: **AND** requires both true, **OR** requires at least one, **NOT** negates. Operator precedence: **NOT** > **AND** > **OR**. Always use parentheses when mixing AND with OR.

Class Example 3 — Female employees in Engineering:

```
SELECT EmpName, Gender, DeptName
FROM Employee
WHERE Gender = 'F' AND DeptName = 'Engineering';
```

Class Example 4 — Employees in Engineering or Research:

```
SELECT EmpName, DeptName
FROM Employee
WHERE DeptName = 'Engineering' OR DeptName = 'Research';
```

Class Example 5 — Female employees in Engineering OR anyone in Research:

```
SELECT EmpName, Gender, DeptName
FROM Employee
WHERE (Gender = 'F' AND DeptName = 'Engineering')
OR DeptName = 'Research';
```

Why parentheses matter

Without parentheses, *Gender = 'F' AND DeptName = 'Engineering' OR DeptName = 'Research'* is evaluated as *(Gender='F' AND DeptName='Engineering') OR DeptName='Research'* by default — which happens to be what we want here. But change the intent slightly and the result becomes wrong silently. Always parenthesise to make intent explicit.

Class Example 6 — Using NOT to invert a condition:

```
SELECT EmpName, DeptName
FROM Employee
WHERE NOT DeptName = 'Sales';
```

4.3 BETWEEN — Range Checks

BETWEEN low AND high is shorthand for `>= low AND <= high` — **both ends are inclusive**. Works on numbers, dates, and even strings (alphabetical order).

Class Example 7 — Salaries between 80,000 and 100,000:

```
SELECT EmpName, Salary
FROM Employee
WHERE Salary BETWEEN 80000 AND 100000;
```

Class Example 8 — Employees hired during 2019:

```
SELECT EmpName, HireDate
FROM Employee
WHERE HireDate BETWEEN '2019-01-01' AND '2019-12-31';
```

Class Example 9 — Combine BETWEEN with NOT:

```
SELECT EmpName, Salary
FROM Employee
WHERE Salary NOT BETWEEN 70000 AND 90000;
```

4.4 IN — Match Against a List

IN (...) tests whether a column matches any value from a list. Cleaner than chaining many ORs. Equally, NOT IN (...) excludes values in the list.

Class Example 10 — Employees in three specific departments:

```
-- Verbose version
SELECT EmpName, DeptName FROM Employee
WHERE DeptName = 'Engineering' OR DeptName = 'Finance' OR DeptName = 'Sales';

-- Cleaner equivalent
SELECT EmpName, DeptName FROM Employee
WHERE DeptName IN ('Engineering', 'Finance', 'Sales');
```

Class Example 11 — Excluding specific cities:

```
SELECT EmpName, City
FROM Employee
WHERE City NOT IN ('Lahore', 'Karachi');
```

4.5 LIKE — Pattern Matching

LIKE matches text patterns using two wildcards: % for any sequence of characters (including empty), and _ for exactly one character. MySQL's LIKE is case-insensitive by default for typical text columns.

Pattern	Matches
'A%'	Any string starting with 'A' (Ali Khan, Ayesha Noor, Adeel Akhtar)
'%a'	Any string ending with 'a' (Hamza Raza, etc.)
'%ali%'	Any string containing 'ali' (Ali Khan, Ayesha Noor, Adeel Akhtar)
'_a%'	Second letter is 'a' (Sara Iqbal, Hamza Raza, Maira Javed, Sana Malik, Talha Hussain)
'%Engineer%'	Job titles containing 'Engineer'

Class Example 12 — Names starting with 'A':

```
SELECT EmpName FROM Employee WHERE EmpName LIKE 'A%';
```

Class Example 13 — Job titles containing the word 'Engineer':

```
SELECT EmpName, JobTitle FROM Employee WHERE JobTitle LIKE '%Engineer%';
```

Class Example 14 — Negation:

```
-- Names that do NOT start with 'A'  
SELECT EmpName FROM Employee WHERE EmpName NOT LIKE 'A%';
```


4.6 IS NULL — Handling Missing Values

NULL means *unknown*. You cannot compare NULL using `=`; `City = NULL` always evaluates to UNKNOWN, never TRUE. Use **IS NULL** or **IS NOT NULL** instead.

Class Example 15 — Wrong vs right way:

```
-- Wrong: returns 0 rows
SELECT EmpName FROM Employee WHERE City = NULL;

-- Correct: returns Nida and Imran
SELECT EmpName FROM Employee WHERE City IS NULL;

-- Correct: returns everyone with a recorded city
SELECT EmpName, City FROM Employee WHERE City IS NOT NULL;
```

The 'unexpected exclusion' problem

When you write `WHERE City != 'Lahore'`, MySQL silently excludes rows where City is NULL — not because they are in Lahore, but because `NULL != 'Lahore'` returns UNKNOWN. To include them, write `WHERE City != 'Lahore' OR City IS NULL`.

4.7 ORDER BY — Sorting Results

ORDER BY sorts the result set. Default is ascending (ASC); use DESC for descending. You can sort by multiple columns — the second column breaks ties from the first.

Class Example 16 — Sort by salary descending:

```
SELECT EmpName, Salary
FROM Employee
ORDER BY Salary DESC;
```

Class Example 17 — Multi-column sort:

```
-- Sort by department name, then by salary highest first within each dept
SELECT EmpName, DeptName, Salary
FROM Employee
ORDER BY DeptName ASC, Salary DESC;
```

4.8 LIMIT — Restricting the Number of Rows

LIMIT n returns only the first *n* rows. Combined with ORDER BY, this is the standard way to find 'top N' results.

Class Example 18 — Top 3 highest paid employees:

```
SELECT EmpName, Salary
FROM Employee
ORDER BY Salary DESC
LIMIT 3;
```

Class Example 19 — 5 most recently hired employees:

```
SELECT EmpName, HireDate
FROM Employee
ORDER BY HireDate DESC
LIMIT 5;
```

5. The Order MySQL Actually Executes a Query

You write a query in this order: SELECT → FROM → WHERE → ORDER BY → LIMIT. MySQL **executes** it in a different order. Knowing this prevents many beginner mistakes — for instance, why you cannot use a SELECT alias inside the WHERE clause.

Step	Clause	What happens
1	FROM	The source table is loaded.
2	WHERE	Rows are filtered. Aliases from SELECT are not yet defined.
3	SELECT	Columns and expressions are computed; aliases assigned.
4	ORDER BY	The final result is sorted (aliases CAN be used here).
5	LIMIT	Only the first N rows are returned.

Practical implication

SELECT Salary * 12 AS AnnualPay FROM Employee WHERE AnnualPay > 1000000; will fail — at the time WHERE runs, the alias *AnnualPay* does not exist yet. Either repeat the expression in WHERE, or wrap the query in a subquery.

6. Lab Tasks

Solve each task in MySQL using the Employee table from section 3. Save all queries in a single file named RollNo_Filters.sql, with a comment line before each answer (e.g. -- Task A1).

Part A — Comparison & Logical Operators

#	Task
A1	List EmpID , EmpName , and Salary of all employees who earn more than 90,000.
A2	Show all employees with salary less than or equal to 75,000. Display EmpName and Salary.
A3	Find every employee who works in Lahore and earns more than 90,000.
A4	List employees in Karachi or Islamabad. Show EmpName and City.
A5	Find female employees who are not in the Engineering department.
A6	Show employees who are male and earn between 70,000 and 90,000 (use AND and comparison operators only — no BETWEEN yet).
A7	List employees who are either Software Engineers or earn more than 100,000.
A8	Find employees who are not in Marketing and not in Sales.

Part B — BETWEEN, IN, LIKE, IS NULL, ORDER BY, LIMIT

#	Task
B1	List employees with salary between 75,000 and 100,000 (inclusive). Sort by salary ascending.
B2	List employees hired between January 2020 and December 2022.
B3	Show employees whose salary is not between 80,000 and 100,000.
B4	List employees whose city is one of: Lahore, Islamabad. Sort by city, then by salary descending.
B5	Find employees in any department except Engineering, Sales, and HR.
B6	Show all employees whose name starts with the letter 'M'. Display EmpName.
B7	Find employees whose name contains the letter 'a' anywhere (case-insensitive).
B8	Find employees whose name ends with 'an'.
B9	Show employees whose job title contains the word 'Engineer' but who do not work in the Engineering department.
B10	List the names of employees who do not have a recorded city.
B11	List employees who have a recorded city, sorted alphabetically by city.
B12	Display the 3 highest paid employees. Show EmpName and Salary.
B13	Display the 5 most recently hired employees.
B14	List the bottom 3 salaries in the company (lowest first).
B15	Show all employees, sorted by department ascending, then by hire date ascending within each department.

Tip

Run each query as you write it and verify the row count matches your expectation. If a query returns fewer rows than expected, check whether NULL values are being silently excluded by your conditions.

7. Assessment Problem (Solve Individually)

Scenario — Online Bookstore. A bookstore stores its catalog in a single Book table. Build it in MySQL, load the provided sample data, and answer the ten questions that follow. *This problem is graded.*

7.1 Setup Script

```
CREATE DATABASE IF NOT EXISTS bookstore_lab;
USE bookstore_lab;

DROP TABLE IF EXISTS Book;

CREATE TABLE Book (
    BookID          INT PRIMARY KEY,
    Title           VARCHAR(80) NOT NULL,
    Author          VARCHAR(60),
    Genre           VARCHAR(30),
    Price           DECIMAL(8,2),
    StockQty        INT,
    PublishedYear    INT,
    Publisher        VARCHAR(40),
    Language        VARCHAR(20)
);

INSERT INTO Book VALUES
(1, 'Pride and Prejudice',      'Jane Austen',      'Fiction', 850, 12, 1813, 'Penguin',
'English'),
(2, 'Emma',                    'Jane Austen',      'Fiction', 900, 8, 1815, 'Penguin',
'English'),
(3, 'Things Fall Apart',      'Chinua Achebe',    'Fiction', 1100, 5, 1958, 'Heinemann',
'English'),
(4, 'Norwegian Wood',         'Haruki Murakami',  'Fiction', 1500, 3, 1987, 'Vintage',
'English'),
(5, 'Kafka on the Shore',     'Haruki Murakami',  'Fiction', 1700, 0, 2002, 'Vintage',
'English'),
(6, 'Ice-Candy-Man',          'Bapsi Sidhwa',     'Fiction', 1200, 15, 1988, 'Penguin',
'English'),
(7, 'The Reluctant Fundamentalist', 'Mohsin Hamid', 'Fiction', 1300, 9,
2007, 'Penguin',      'English'),
(8, 'Exit West',              'Mohsin Hamid',     'Fiction', 1450, 6, 2017, 'Riverhead',
'English'),
(9, 'Atomic Habits',          'James Clear',      'Self-help', 1800, 20, 2018, 'Avery',
'English'),
(10, 'The Power of Habit',     'Charles Duhigg',   'Self-help', 1600, 11, 2012, 'Random
House', 'English'),
(11, 'Sapiens',               'Yuval Harari',     'History', 2200, 7, 2011, 'Harper',
'English'),
(12, 'Rich Dad Poor Dad',     'Robert Kiyosaki',  'Finance', 1100, 25, 1997, 'Plata',
'English'),
(13, 'Aab-e-Hayat',           'Ibn-e-Safi',       'Mystery', 650, 18, 1955, 'Asrar',
'Urdu'),
(14, 'Raja Gidh',             'Bano Qudsia',      'Fiction', 900, 14, 1981, 'Sang-e-Meel',
'Urdu'),
(15, 'Mystery Title',         NULL,                'Mystery', 950, 4, 2020, NULL,
'English');
-- Note: Book 15 has unknown author and unknown publisher (NULL)
```

7.2 Questions

#	Question	Marks
Q1	List all books with a price greater than 1500. Show Title and Price.	5
Q2	Find all books published between 1900 and 2000 . Show Title and PublishedYear, sorted by year.	10
Q3	List books in Fiction or Mystery genre with stock greater than 5.	10
Q4	Find books whose title contains the word 'the' anywhere (case-insensitive). Show Title and Author.	10
Q5	List books whose title starts with the letter 'A' or ends with 't'.	10
Q6	Find books with no recorded author. Show Title.	5
Q7	List books that are out of stock (StockQty = 0) or have unknown publisher.	10
Q8	Show the 3 most expensive books in stock (StockQty > 0).	15
Q9	Display all books written in Urdu, sorted by published year ascending.	10
Q10	List books published before the year 2000 with a price under 1200, sorted by genre and then by title.	15

7.3 Submission

- Push your RollNo_Filters.sql file to your existing GitHub repository in a folder named Lab1-Filters/.
- The file must contain: the setup script (or a comment confirming you ran it), all Lab Task answers (Parts A and B), and all 10 Assessment answers, each labelled with a comment (e.g. -- Q5).
- Share the GitHub link to that folder before the deadline announced in class.

8. Common Mistakes to Avoid

- Comparing to NULL with = or !=. Always use IS NULL / IS NOT NULL.
- Mixing AND and OR without parentheses. The query may run, but return wrong rows silently.
- Forgetting that != 'value' excludes NULL rows. Add OR col IS NULL if you want them included.
- Using LIKE with a leading % wildcard on a large table — it cannot use indexes and will be slow.
- Putting quotes around numeric values: WHERE Salary = '95000' works due to implicit conversion but is bad practice and can hide type errors.
- Using BETWEEN exclusively for dates and forgetting it is *inclusive* on both ends.
- Trying to reference a SELECT alias inside the WHERE clause. The alias does not exist yet at that stage.
- Forgetting ORDER BY when using LIMIT — without sorting, MySQL may return any 3 rows, not the top 3.

9. Appendix — Quick Reference

```
-- BASIC STRUCTURE
SELECT cols
FROM table
WHERE condition
ORDER BY col [ASC|DESC]
LIMIT n;

-- COMPARISON      = != < > <= >=
-- LOGICAL          AND OR NOT      (precedence: NOT > AND > OR)

-- RANGE
WHERE col BETWEEN low AND high      -- inclusive on both ends
WHERE col NOT BETWEEN low AND high

-- LIST
WHERE col IN (v1, v2, v3)
WHERE col NOT IN (v1, v2)

-- PATTERN MATCHING (% = any chars, _ = exactly one char)
WHERE name LIKE 'A%'                -- starts with A
WHERE name LIKE '%a'                -- ends with a
WHERE name LIKE '%mid%'              -- contains mid
WHERE name LIKE '_a%'                -- second char is a
WHERE name NOT LIKE 'A%'

-- NULL HANDLING
WHERE col IS NULL
WHERE col IS NOT NULL
-- NEVER: WHERE col = NULL (always returns 0 rows)

-- SORTING (multi-column)
ORDER BY DeptName ASC, Salary DESC

-- TOP N
ORDER BY Salary DESC
LIMIT 5;
```

End of Lab Manual #1 — SQL Filters. Next lab: SQL Joins.